



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e de Informática

Análise Crítica do Artigo “Leaking Information Through Cache LRU States in Commercial Processors and Secure Caches”

Felipe Gabriel de Carvalho
Felipe Silva Barros
Gabriel Henrique de Moraes
Gustavo Henrique dos Santos Dias
Thiago Henrique Gomes Feliciano

Resumo

Políticas de substituição Least-Recently Used (LRU) são usadas em caches de processadores para decidir qual linha deve ser removida quando não há espaço. Canais laterais e encobertos em cache já são conhecidos, mas normalmente dependem de cache misses, o que facilita sua detecção por meio de contadores de desempenho. Este artigo investiga se o próprio estado de substituição LRU, que é atualizado a cada acesso, inclusive em cache hits, pode ser usado para transmitir informações sem provocar misses. Para isso, são propostos dois protocolos de comunicação, com e sem memória compartilhada, avaliados em processadores Intel e AMD usando hyper-threading e fatiamento de tempo. Também são testados um ataque do tipo Spectre e dois designs de cache seguro simulados no gem5. Os resultados mostram que esse canal é viável na prática e pode alcançar altas taxas de transmissão em processadores reais. Além disso, os caches seguros avaliados não protegem adequadamente o estado de substituição. Conclui-se que proteger apenas os dados e as tags do cache não é suficiente. O próprio estado de substituição LRU também precisa ser protegido para garantir a segurança dos caches.

Palavras-chave: Cache; LRU; política de substituição; segurança de hardware; execução especulativa.

Abstract

Least-Recently Used (LRU) replacement policies are used in processor caches to decide which line should be removed when there's no space. Side and covert channels in cache are already known, but they usually rely on cache misses, which makes them easier to detect with performance counters. This article investigates whether the LRU replacement state itself, which is updated on every access, even on cache hits, can be used to transmit information without causing misses. To do this, two communication protocols are proposed, with and without shared memory, tested on Intel and AMD processors using hyper-threading and time slicing. A Spectre-type attack and two secure cache designs simulated in gem5 are also tested. The results show that this channel is practical and can achieve high transmission rates on real processors. Moreover, the evaluated secure caches don't properly protect the replacement state. It is concluded that just protecting the data and cache tags is not enough. The LRU replacement state itself also needs to be protected to ensure cache security.

Keywords: Cache; LRU; replacement policy; hardware security; speculative execution.

1 INTRODUÇÃO AO TEMA

A hierarquia de memória é um dos componentes mais críticos do desempenho em arquiteturas modernas, e o cache de dados de primeiro nível (L1) desempenha papel central por estar diretamente acoplado ao pipeline do processador. Para decidir qual linha de cache deve ser removida quando ocorre uma falta (miss) em um conjunto associativo, os processadores empregam políticas de substituição, sendo a política Least-Recently Used (LRU), e suas variantes de custo reduzido, como Tree-PLRU e Bit-PLRU, as mais amplamente adotadas por oferecerem boas taxas de acerto com overhead de hardware relativamente baixo.

Nos últimos anos, consolidou-se uma extensa literatura sobre canais laterais (side channels) e canais encobertos (covert channels) em caches, nos quais o tempo de acesso à memória é usado como sinal para inferir o comportamento de outro processo ou vítima. A grande maioria desses ataques, como Flush+Reload e Prime+Probe, depende de o remetente (sender) provocar uma falta de cache (cache miss) para alterar o estado observável pelo receptor. No artigo de (Xiong; Katzenbeisser; Szefer, 2021) escolhido por este grupo rompe com essa premissa: demonstra que o próprio estado de substituição LRU, atualizado a cada acesso, constitui por si só um canal de comunicação temporal viável, mais furtivo e potencialmente mais rápido do que os canais tradicionais baseados em falta de cache.

Este trabalho apresenta uma revisão crítica do artigo-base, sua contextualização frente a quatro trabalhos correlatos e uma proposta inicial de oportunidade de pesquisa fundamentada nas limitações identificadas.

2 REVISÃO E ANÁLISE CRÍTICA DO ARTIGO BASE

2.1 Síntese das contribuições

O trabalho de Xiong, Katzenbeisser e Szefer (2021) apresenta uma nova forma de canal de temporização baseada no estado da política de substituição LRU/PLRU da cache, demonstrando que tanto cache hits quanto cache misses podem modificar esse estado e permitir que um receptor obtenha informações por meio de medições de tempo. Os autores desenvolvem dois protocolos, um com memória compartilhada e outro sem memória compartilhada, e demonstram sua aplicação em processadores Intel e AMD, considerando cenários de hyper-threading e time-slicing.

Os resultados mostram taxas de transmissão de até aproximadamente 600 Kbps por conjunto de cache no cenário de hyper-threading. Além disso, o trabalho demonstra a utilização do canal como mecanismo de divulgação de informações em ataques Spectre v1 e avalia sua eficácia contra as caches seguras Partition-Locked e Random Fill por meio de simulações no gem5

2.2 Pontos fortes

Um dos principais pontos fortes do artigo está na validação experimental do mecanismo proposto em diferentes processadores e condições de compartilhamento da cache, demonstrando que o canal não depende exclusivamente de uma arquitetura ou configuração específica. A utilização de métricas como Hamming Distance e Edit Distance também permite avaliar os erros de transmissão de maneira quantitativa, sendo o Hamming Distance destacado pelos autores por sua maior compatibilidade com códigos de correção de erros utilizados na prática. Outro aspecto relevante é a análise de segurança realizada sobre diferentes mecanismos de proteção, especialmente Partition-Locked e Random Fill, mostrando que a proteção dos dados e das tags da cache não é necessariamente suficiente quando o estado de substituição permanece compartilhado.

2.3 Limitações e lacunas identificadas

Apesar da qualidade da contribuição, a leitura crítica do artigo revela limitações relevantes, como:

O artigo base possui um escopo restrito ao cache L1, os próprios autores reconhecem a possibilidade de exploração do canal em outros níveis da hierarquia de memória, como L2 e LLC, porém não apresentam uma demonstração experimental desses cenários. Dessa forma, a

afirmação de que L2 e LLC seriam “mais fáceis de proteger” por meio de técnicas de mitigação já existentes permanece sem validação quantitativa.

Existe uma cobertura limitada de caches seguros, apenas PL cache e RF cache são avaliados experimentalmente. Defesas baseadas em randomização do mapeamento endereço–conjunto, como ScatterCache, CEASER, CEASER-S e caches skewed, são apenas discutidas qualitativamente, sem simulação. Assim, falta uma avaliação quantitativa de sua eficácia contra ataques baseados em eviction sets e de seus respectivos custos e limitações.

Os processadores avaliados, Sandy Bridge, Skylake e Zen 1, já não representam as microarquiteturas comerciais mais recentes. Portanto, constituem plataformas de hardware datadas em relação aos processadores atuais, que incorporam hierarquias de cache mais sofisticadas, políticas de substituição mais adaptativas e mecanismos de coerência e gerenciamento de granularidade potencialmente mais complexos.

Além disso, o próprio artigo indica, na Seção 5.1.2, que o Algoritmo 2 (sem memória compartilhada) não produziu sinal observável em compartilhamento time-sliced nos processadores AMD testados, uma limitação prática relevante que não é aprofundada.

3 DISCUSSÃO DOS TRABALHOS CORRELATOS

Foram escolhidos quatro trabalhos relacionados, publicados principalmente nos últimos três anos, que abordam o tema do artigo-base por diferentes perspectivas. Eles tratam da segurança de caches, da influência das políticas de substituição em caches randomizados, de arquiteturas que evitam o fetch/replace determinístico e, por fim, de uma revisão geral sobre canais temporais em microarquiteturas

3.1 Generic and Automated Drive-by GPU Cache Attacks from the Browser

O trabalho de (Giner et al., 2024) se relaciona com o artigo-base porque também explora a política de substituição da cache como forma de vaziar informação, mas aplica essa ideia em um cenário bem diferente: em vez de atacar a cache L1 de CPUs comerciais usando código nativo, os autores atacam a cache L2 de GPUs a partir do navegador, usando apenas a API WebGPU. A diferença principal está no mecanismo usado: (Xiong; Katzenbeisser; Szefer, 2021) mostram que dá para ler o estado do LRU só com acessos que resultam em cache hit, sem precisar provocar uma falta de cache. Já o ataque de Giner et al. (2024) usa o método clássico de Prime+Probe, que ainda depende de a vítima causar misses para que o receptor consiga observar algo. Ou seja, os dois trabalhos se complementam: o artigo-base quebra a ideia de que é preciso causar um miss para vaziar informação pelo estado de substituição, enquanto (Giner et al., 2024) mostram que, mesmo em um ambiente muito mais limitado como o navegador (sem timers de alta precisão e sem memória compartilhada entre processos), ainda é possível

reconstruir os ataques tradicionais criando as primitivas necessárias por software, como um timer usando threads, detecção do tamanho da cache e construção de conjuntos de evicção.

Outro ponto em comum é que os dois ataques dependem do comportamento previsível de uma política de substituição parecida com LRU. No artigo-base, esse comportamento é usado para codificar e decodificar mensagens observando um único bit por acesso. Em (Giner et al., 2024), esse mesmo tipo de comportamento é aproveitado tanto para construir conjuntos de evicção em paralelo quanto para o ataque de recuperação de chave AES, em que remover um único endereço de um grupo já revela um conjunto de evicção inteiro, por causa do que os autores chamam de "efeito cascata de evicção". Isso reforça uma das limitações já discutidas na análise crítica do artigo-base: o vazamento pelo estado de substituição não é um problema isolado de um fabricante específico ou de um nível da hierarquia de memória, mas sim um problema estrutural, que aparece sempre que uma política de substituição previsível é compartilhada entre partes que não deveriam trocar informação — seja na L1 de CPUs (caso do artigo-base) ou na L2 de GPUs (Giner et al., 2024).

Por fim, o trabalho de (Giner et al., 2024) também amplia o tipo de atacante considerado. Enquanto (Xiong; Katzenbeisser; Szefer, 2021) assumem um atacante que já tem código nativo rodando no mesmo núcleo da vítima (por hyper-threading ou compartilhamento por tempo), (Giner et al., 2024) mostram que um atacante remoto, que só precisa fazer a vítima visitar uma página maliciosa, consegue montar canais de temporização e covert channels parecidos, chegando a até 10,9 kB/s, usando apenas JavaScript e WGSF. Essa diferença mostra uma lacuna parecida com a apontada na crítica ao artigo-base: falta avaliar esses canais baseados no estado de substituição em cenários mais realistas e com atacantes menos privilegiados, como dentro de ambientes restritos de navegador, algo que os autores do artigo-base não testam.

3.2 Random and Safe Cache Architecture to Defeat Cache Timing Attacks

O artigo de (Hu; Lee, 2024) propõe uma mudança de paradigma mais radical do que as defesas discutidas por (Xiong; Katzenbeisser; Szefer, 2021), em vez de apenas embaralhar o endereço de preenchimento como faz a Random Fill (RF) cache, que já foi testada e considerada insuficiente no trabalho de (Xiong; Katzenbeisser; Szefer, 2021), a arquitetura RaS dissocia completamente o preenchimento do cache do acesso demandado pelo processador e abandona por completo a política de substituição determinística (LRU/PLRU), adotando substituição aleatória (RandRP). O conjunto de cache afetado por uma evicção deixa de guardar qualquer relação com o conjunto acessado pela vítima, e o preenchimento passa a ocorrer a partir de um Safe History Buffer (SHB) de endereços já autorizados, com seleção aleatória de linha dentro de uma janela, não mais do endereço realmente requisitado.

A validação é feita totalmente por simulação no gem5, usando dois protótipos: o RaS-Spec, voltado para ataques especulativos, e o RaS+, que também protege contra ataques não especulativos. Os dois são testados contra ataques como Spectre v1, Prime-Probe, Flush-Reload,

Evict-Time e Cache-Collision, usando o AES-128. Além disso, o desempenho é comparado com outras defesas, como InvisiSpec, CleanupSpec, MuonTrap, GhostMinion e a própria RF cache.

Enquanto (Xiong; Katzenbeisser; Szefer, 2021) mostra, em hardware real, que políticas de substituição como LRU e PLRU podem vaziar informações mesmo quando ocorrem hits, (Hu; Lee, 2024) consideram essa vulnerabilidade como algo já conhecido. A proposta deles é atacar o problema diretamente, eliminando o comportamento determinístico da política de substituição, em vez de apenas tentar esconder ou dificultar a previsão dos endereços que são preenchidos no cache.

A RaS é uma evolução das defesas por randomização, como a RF cache, mas vai além ao randomizar a própria política de substituição do cache. Isso pode impedir que o atacante descubra o estado do cache, tornando a política sem estado. Porém, ainda não está comprovado se a RaS realmente elimina o canal hit-only baseado em LRU/PLRU. Isso porque os autores não testaram a RaS diretamente contra esse tipo específico de ataque. Assim, embora a randomização da substituição possa impedir que o atacante descubra o estado do cache, ainda não é possível afirmar que ela elimina completamente esse vazamento.

3.3 Analysis of Secure Caches Using a Three-Step Model for Timing-Based Attacks

O artigo de (Deng; Xiong; Szefer, 2019) propõe um modelo sistemático para identificar e analisar vulnerabilidades relacionadas a ataques de temporização em memórias cache. Diferentemente de trabalhos que apresentam um ataque específico ou uma arquitetura de cache projetada para impedir determinado canal lateral, os autores buscam estabelecer uma metodologia capaz de enumerar diferentes formas pelas quais o estado do cache pode ser alterado e posteriormente observado pelo atacante. O modelo considera não apenas acessos realizados pelo próprio processador, mas também alterações decorrentes do protocolo de coerência entre diferentes núcleos, além de ataques convencionais e ataques associados à execução especulativa.

A principal contribuição do trabalho é o chamado modelo de três etapas, no qual uma vulnerabilidade de timing é representada por uma sequência de ações capazes de modificar o estado do cache e posteriormente produzir uma diferença observável pelo atacante. Para cada etapa, os autores enumeram diferentes estados possíveis de um bloco de cache, considerando se a operação foi realizada pelo atacante ou pela vítima, se os dados pertencem a regiões de memória distintas ou relacionadas e se a alteração ocorreu devido a um acesso à memória ou a uma operação de invalidação. Dessa forma, o modelo permite gerar sistematicamente diferentes combinações de estados que podem resultar em um canal lateral baseado em tempo.

A partir desse modelo, os autores analisam diferentes arquiteturas de cache consideradas seguras e verificam quais tipos de vulnerabilidades cada mecanismo consegue impedir. A análise não se limita a ataques tradicionais realizados por meio de acessos ou instruções de flush,

incorporando também situações nas quais o estado do cache é modificado por outro núcleo através do protocolo de coerência. Essa abordagem é importante porque demonstra que uma defesa aparentemente eficaz contra um determinado tipo de ataque pode continuar vulnerável quando o atacante utiliza uma forma diferente de alterar ou observar o estado da cache.

Um ponto relevante do trabalho é a distinção entre vulnerabilidades fortes, fracas e ineficazes. Uma vulnerabilidade é considerada mais significativa quando as ações da vítima produzem estados suficientemente distintos para que o atacante consiga inferir informações por meio da medição do tempo de acesso. Já nos casos classificados como fracos, existe alguma relação entre o comportamento da vítima e a observação realizada pelo atacante, porém a distinção pode ser menos confiável. Essa classificação permite avaliar não apenas se uma arquitetura é ou não vulnerável, mas também o grau em que determinada configuração facilita a exploração do canal lateral.

A metodologia apresentada por (Deng; Xiong; Szefer, 2019) também possui uma relação direta com trabalhos posteriores dos mesmos autores. Em particular, o modelo evidencia que mecanismos de proteção podem eliminar determinados caminhos de ataque sem necessariamente eliminar todas as possibilidades de observação do estado interno da cache. Essa questão se torna especialmente relevante no trabalho de (Deng; Xiong; Szefer, 2019), no qual é demonstrado que informações podem ser extraídas a partir dos estados de substituição LRU em processadores comerciais, inclusive em situações nas quais não ocorre uma mudança convencional de hit para miss.

Enquanto (Deng; Xiong; Szefer, 2019) concentram-se em criar uma metodologia abrangente para enumerar e classificar possíveis canais de timing, (Deng; Xiong; Szefer, 2019) demonstram experimentalmente uma classe específica de vazamento relacionada ao estado LRU da cache. Assim, o primeiro trabalho pode ser interpretado como uma abordagem sistemática para compreender o espaço de vulnerabilidades, enquanto o segundo demonstra como uma dessas características internas da arquitetura pode ser transformada em um canal lateral explorável em processadores reais.

Entretanto, uma limitação importante do artigo de (Deng; Xiong; Szefer, 2019) é que grande parte da análise das arquiteturas de cache é realizada de maneira sistemática e manual a partir do modelo proposto, não constituindo uma validação experimental abrangente em hardware comercial. Dessa forma, a identificação de uma vulnerabilidade pelo modelo não significa necessariamente que sua exploração apresente o mesmo desempenho ou confiabilidade em um processador real. Aspectos de implementação, ruído temporal, políticas específicas do fabricante e outros mecanismos internos podem alterar significativamente a capacidade prática de exploração.

Além disso, o modelo é baseado na representação dos estados e transições relevantes do cache, o que permite uma análise bastante abrangente, mas também introduz uma abstração em relação ao comportamento completo dos processadores modernos. Características não contempladas ou implementadas de maneira diferente em arquiteturas reais podem produzir novos comportamentos que não sejam completamente representados pelo modelo. Isso demonstra que

uma metodologia formal de classificação é útil para encontrar potenciais canais, mas deve ser complementada por experimentos em hardware para determinar a relevância prática de cada vulnerabilidade.

Dessa forma, o principal mérito do artigo está em transformar a análise de ataques de temporização em um problema sistemático, permitindo comparar diferentes mecanismos de proteção sob uma mesma metodologia. O trabalho também evidencia que a segurança de uma cache não pode ser avaliada apenas considerando ataques conhecidos, uma vez que diferentes formas de modificar e observar seu estado podem produzir canais laterais distintos. Apesar das limitações relacionadas à abstração e à ausência de uma validação experimental completa em processadores comerciais, o modelo de três etapas fornece uma base importante para trabalhos posteriores sobre ataques e defesas de cache.

Em conjunto com trabalhos posteriores, como (Deng; Xiong; Szefer, 2019), o artigo demonstra uma evolução importante na área: inicialmente, busca-se estabelecer uma forma sistemática de identificar os possíveis canais de temporização; posteriormente, características específicas das arquiteturas comerciais são exploradas para demonstrar novos vazamentos. Assim, embora o modelo de três etapas seja capaz de organizar e identificar diferentes classes de vulnerabilidades, os resultados posteriores mostram que a análise de segurança de caches deve continuar sendo acompanhada por avaliações experimentais, especialmente diante de mecanismos internos como LRU/PLRU, coerência entre núcleos e execução especulativa.

3.4 Prime+Retouch: When Cache is Locked and Leaked

O artigo “Prime+Retouch: When Cache is Locked and Leaked”, publicado em 2024 por (Lee, Sang e Kim), apresenta uma contribuição diretamente relacionada ao trabalho de (Xiong; Katzenbeisser; Szefer, 2021), pois ambos demonstram que o vazamento de informações em caches não depende exclusivamente da observação de cache misses ou da substituição direta dos dados da vítima. O artigo de (Xiong; Katzenbeisser; Szefer, 2021) mostra que o próprio estado da política de substituição LRU/PLRU pode funcionar como um canal de vazamento, onde qualquer acesso realizado pelo emissor pode modificar o estado, e o receptor consegue inferir essa alteração por meio de medições de temporização.

O trabalho de (Lee; Sang; Kim, 2024) amplia essa perspectiva ao demonstrar que os metadados associados à política de substituição também podem ser explorados para inferir atividades da vítima. O ataque Prime+Retouch consegue obter informações sem provocar a perda dos dados da vítima, explorando o fato de que embora os dados possam estar protegidos ou não sejam compartilhados diretamente, os metadados utilizados pela política de substituição permanecem compartilhados. Os autores utilizaram arquiteturas Intel x86 e Apple M1 para demonstrarem o ataque e a técnica. Dessa forma, os dois trabalhos partem de uma observação fundamental em comum: o estado da política de substituição da cache constitui uma superfície de ataque independente do conteúdo armazenado nas linhas de cache. (Xiong; Katzenbeisser;

Szefer, 2021) demonstram essa propriedade utilizando o estado LRU/PLRU como um canal de temporização, enquanto (Lee; Sang; Kim, 2024) mostram que os metadados da política podem ser utilizados para realizar ataques mesmo quando mecanismos de defesa procuram impedir a perda dos dados sensíveis.

Com isso, podemos considerar o trabalho de (Lee; Sang; Kim, 2024) uma evolução do mesmo problema de segurança. Enquanto (Xiong; Katzenbeisser; Szefer, 2021). evidenciam que acessos que resultam em hits ainda podem modificar informações relevantes para a política de substituição, Prime+Retouch mostra que essa informação pode continuar sendo explorada em arquiteturas mais recentes e mesmo diante de mecanismos de proteção que impedem a substituição dos dados da vítima. No trabalho de (Xiong; Katzenbeisser; Szefer, 2021), essa limitação também aparece no experimento com a Partition-Locked cache: mesmo com a linha protegida contra substituição, seu acesso continua modificando o estado LRU, permitindo que o receptor infira o padrão de acesso.

Tabela 1 – Comparação entre os trabalhos de Xiong et al. (2021) e Lee et al. (2024).

Aspecto	Xiong et al. (2021)	Lee et al. (2024)
Ideia central	Exploração do estado LRU/PLRU como fonte de vazamento de informação	Exploração de metadados associados à política de substituição do cache
Mecanismo	Os acessos modificam o estado da política de substituição, permitindo inferir atividades da vítima	Os metadados do mecanismo de substituição são observados e utilizados para inferir acessos da vítima
Eviction da vítima	Não é necessária para o canal baseado somente em cache hits	Não é necessária para realizar o ataque
Memória compartilhada	O ataque pode funcionar sem memória compartilhada	O vazamento pode ocorrer por meio de memória não compartilhada
Arquiteturas avaliadas	Intel e AMD	Intel x86 e Apple M1
Defesas analisadas	Partition-Locked e Random Fill	Mecanismos de locking e pre-fetching
Relação com o problema	Demonstra que o estado da política LRU/PLRU pode atuar como canal lateral	Demonstra que informações auxiliares da política de substituição permanecem exploráveis mesmo quando o cache é protegido

Assim, o artigo de 2024 é relevante para o artigo-base porque não apenas utiliza um princípio semelhante, mas mostra que a superfície de ataque identificada por (Xiong; Katzenbeisser; Szefer, 2021) permanece relevante em arquiteturas modernas e contra mecanismos de defesa mais recentes

4 PROPOSTA INICIAL DE OPORTUNIDADE DE PESQUISA

4.1 Fundamentação da lacuna

Apesar de demonstrar a viabilidade do canal baseado nos estados LRU e PLRU, o artigo concentra sua avaliação no cache L1. Os autores mencionam a possibilidade de extensão para os níveis L2 e LLC, mas não apresentam experimentos que comprovem o comportamento do canal nesses níveis. Como L2 e LLC possuem maior capacidade, associatividade e diferentes formas de compartilhamento, ainda não é possível afirmar se o canal apresentaria a mesma capacidade, taxa de erro ou facilidade de exploração.

Outra lacuna está na quantidade limitada de mecanismos de proteção avaliados. Apenas PL cache e RF cache são testados experimentalmente, enquanto propostas como ScatterCache, CEASER, CEASER-S e caches skewed são discutidas apenas de forma qualitativa. Dessa forma, ainda falta uma comparação quantitativa capaz de verificar se a randomização impede o canal ou somente dificulta sua construção.

Também é necessário considerar que os processadores utilizados nos experimentos, como Sandy Bridge, Skylake e Zen 1, não representam as microarquiteturas comerciais mais recentes. Processadores atuais possuem hierarquias de cache mais complexas e políticas de substituição adaptativas, o que pode alterar tanto a capacidade do canal quanto sua resistência a mecanismos modernos de proteção.

Por fim, o protocolo sem memória compartilhada não produziu um sinal observável em determinados experimentos time-sliced realizados em processadores AMD. O artigo apresenta esse resultado, mas não identifica com precisão sua causa. Portanto, a ausência do sinal pode estar relacionada à política de substituição, ao escalonamento do sistema operacional, ao way predictor ou a outras características internas da arquitetura.

Essas limitações justificam uma investigação mais ampla do canal em diferentes níveis de cache, processadores atuais e mecanismos de randomização. Também justificam uma análise específica do comportamento observado nas plataformas AMD, permitindo compreender se a falha representa uma limitação do protocolo ou uma característica arquitetural capaz de dificultar o vazamento.

4.2 Oportunidade de Pesquisa proposta

A partir das limitações identificadas, propõem-se quatro frentes de pesquisa futura, que podem ser integradas em um único programa experimental.

1. Extensão do canal LRU para L2 e LLC. Investigar o canal LRU/PLRU nos níveis L2 e LLC, utilizando simuladores como gem5 e ChampSim e processadores comerciais. Os algoritmos do estudo poderiam ser adaptados a políticas modernas, como RRIP e Hawkeye,

avaliando taxa de detecção, capacidade do canal e taxa de erro. Também seria relevante verificar a hipótese de que L2/LLC são mais fáceis de proteger diante de técnicas de construção de eviction sets, especialmente em ataques cross-core.

2. Avaliação de defesas baseadas em randomização. Avaliar mecanismos como ScatterCache, CEASER, CEASER-S e caches skewed frente ao canal LRU/PLRU. O objetivo seria verificar se a randomização impede efetivamente a formação do canal ou se técnicas adaptativas de descoberta de conjuntos permitem sua reconstrução parcial. Deve-se também quantificar o overhead de desempenho e estabelecer um comparativo entre segurança e desempenho.

3. Reprodução em microarquiteturas contemporâneas. Reavaliar o canal em processadores atuais, como Intel Alder Lake/Raptor Lake e AMD Zen 4/5, considerando hierarquias de cache mais complexas, políticas de substituição adaptativas e mecanismos modernos de mitigação de canais laterais. Essa análise permitiria verificar se a capacidade de transmissão, originalmente reportada em até 600 Kbps, permanece significativa e se as novas arquiteturas oferecem proteção parcial.

4. Investigação da falha do Algoritmo 2 em time-sliced sharing. Investigar a ausência de sinal observada em processadores AMD, analisando fatores como granularidade dos contadores, comportamento do utag/way-predictor, escalonamento do sistema operacional e tamanho do quantum. Também poderiam ser avaliados ajustes nos parâmetros do canal e técnicas avançadas de filtragem de ruído, buscando recuperar um canal funcional sem memória compartilhada, cenário particularmente relevante para a segurança.

5 CONSIDERAÇÕES FINAIS

Este trabalho apresentou uma análise crítica do artigo “Leaking Information Through Cache LRU States in Commercial Processors and Secure Caches”, destacando suas contribuições, seus pontos fortes e suas principais limitações. O artigo demonstra que os estados das políticas de substituição LRU e PLRU podem ser utilizados como canais de comunicação, inclusive em situações nas quais os acessos produzem cache hits.

A validação em processadores Intel e AMD mostra que o problema possui relevância prática e não se limita a uma possibilidade teórica. Além disso, os testes realizados em caches seguros demonstram que proteger somente os dados, as tags ou o mapeamento dos conjuntos não é suficiente. Os metadados responsáveis pela substituição das linhas também devem ser considerados no modelo de segurança.

Entretanto, a concentração dos experimentos no cache L1, o uso de microarquiteturas mais antigas e a avaliação limitada de mecanismos de proteção deixam questões ainda não respondidas. Também permanece sem uma explicação conclusiva a ausência de sinal do protocolo sem memória compartilhada em determinados cenários com processadores AMD.

Diante disso, a oportunidade de pesquisa proposta busca ampliar os experimentos para os níveis L2 e LLC, avaliar mecanismos de randomização, reproduzir o canal em processadores

atuais e investigar as limitações observadas em plataformas AMD. Essas análises podem contribuir para uma compreensão mais completa dos canais baseados em políticas de substituição e para o desenvolvimento de caches mais seguros.

REFERÊNCIAS

DENG, S.; XIONG, W.; SZEFER, J. Analysis of secure caches using a three-step model for timing-based attacks. **Journal of Hardware and Systems Security**, v. 3, p. 397–425, 2019.

GINER, L. et al. Generic and automated drive-by gpu cache attacks from the browser. *In: **Proceedings of the 19th ACM Asia Conference on Computer and Communications Security***. [S.l.]: ACM, 2024. p. 128–140.

HU, G.; LEE, R. B. **Random and Safe Cache Architecture to Defeat Cache Timing Attacks**. 2024.

LEE, J.; SANG, F.; KIM, T. **Prime+Retouch: When Cache is Locked and Leaked**. 2024.

XIONG, W.; KATZENBEISSER, S.; SZEFER, J. Leaking information through cache LRU states in commercial processors and secure caches. **IEEE Transactions on Computers**, 2021.